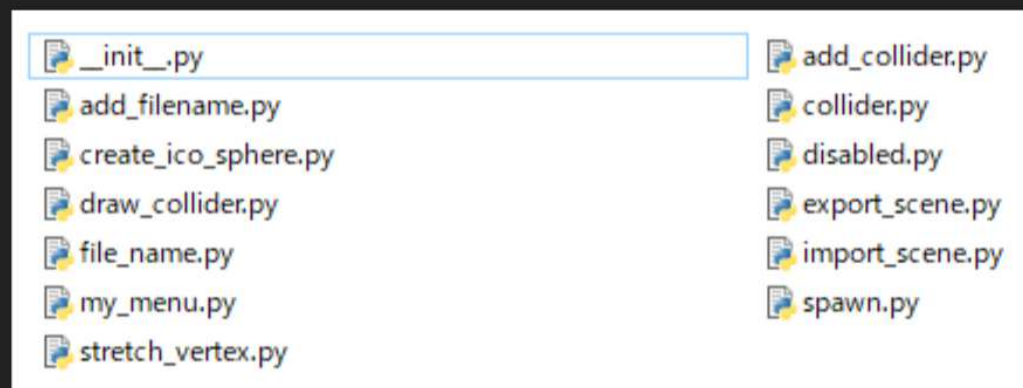


今回の目標

日本工学院専門学校
デザインカレッジ

- レベルエディタのpythonスクリプトを複数ファイルに分割する



TL1

02_02.モジュール分離

日本工学院専門学校
デザインカレッジ

docs.google.com – 全画面表示を終了するには、Esc を押します

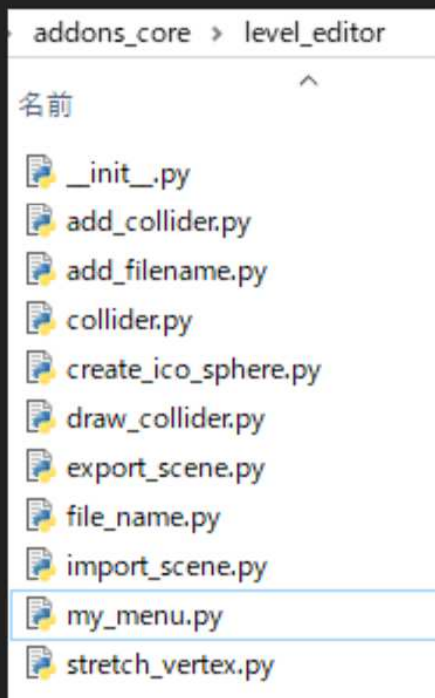
TL1

02_02.モジュール分離

日本工学院専門学校
デザインカレッジ

モジュール化を進める

日本工学院専門学校
デザインカレッジ



レベルエディタのように
機能が複雑であるほど、きっちりファイル分割して
整理することの重要度が増す。

綺麗に整理してから次以降の機能拡張を始めよう。

※左図は分け方の1例であって、
必ずこの通りに分けろという意味ではない。

モジュール化を進める

日本工学院専門学校
デザインカレッジ

```
# Blenderに登録するクラスリスト
classes = (
    MYADDON_OT_stretch_vertex,
    MYADDON_OT_create_ico_sphere,
    MYADDON_OT_import_scene,
    MYADDON_OT_export_scene,
    TOPBAR_MT_my_menu,
    MYADDON_OT_add_filename,
    OBJECT_PT_file_name,
    MYADDON_OT_add_collider,
    OBJECT_PT_collider,
)
```

ここまで上手くいったら、他の機能もモジュール化を進めて、`__init__.py` のスリム化を図ろう。

1つのスクリプトファイルに複数のクラスを定義することも可能だがC++と同じでまとめすぎるとチーム内の分担や保守性に問題が出る。

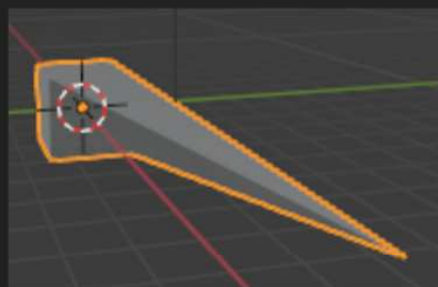
「一つの機能に関するオペレータクラスとメニュークラス、パネルクラスは1ファイルにまとめてよい」など、ルールを決めて運用しよう。

とりあえず1ファイル1クラスにしてもよい。

モジュール化を進める

動作確認

日本工学院専門学校
デザインカレッジ



びろーん

「頂点を伸ばす」機能のモジュール分割が完了した。
この時点で元通りの動作をしていることを確認しておく。

自作モジュールのimport

日本工学院専門学校
デザインカレッジ

__init__.py -level_editor

```
# アドオン情報
bl_info = {
    ...
}

# モジュールのインポート
...
+ from .stretch_vertex import MYADDON_OT_stretch_vertex

# クラスやグローバル関数の定義
...

def register():
    ...

def unregister():
    ...
```

移植によって __init__.py から
頂点を伸ばすオペレータがなくなった状態なので、`import` で結合する。

`from` でファイルパスを指定し、
クラス名や関数名を指定して `import` する。

この場合はオペレータのクラス名を指定する。

これで __init__.py から見ても
元通り MYADDON_OT_stretch_vertex クラスが存在することになる。

importの追加

日本工学院専門学校
デザインカレッジ

stretch_vertex.py -level_editor

+

```
import bpy
```

```
#オペレータ 頂点を伸ばす
```

```
class MYADDON_OT_stretch_vertex(bpy.types.Operator):  
    ...
```

貼り付けただけの状態だと、
stretch_vertex.py 側にエラーが出るので
修正する。

具体的には移植したクラスで使っている
python や Blender Python API の
モジュールがないとエラーになるので、
必要なモジュールのimportを追加してやる。

スクリプト言語なので
エラーは本来実行するまで分からないが
VSCode などを使っていると
実行前状態でも問題点を
指摘してくれたりする。

クラスの移植

日本工学院専門学校
デザインカレッジ

stretch_vertex.py -level_editor

#オペレータ 頂点を伸ばす

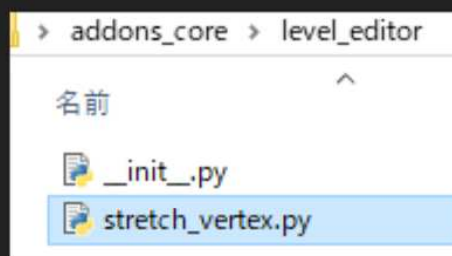
```
class MYADDON_OT_stretch_vertex(bpy.types.Operator):  
    ...
```

__init__.py にある
頂点を伸ばすオペレータクラスを
丸ごと切り取って

stretch_vertex.py に貼り付ける。

最小のモジュール

日本工学院専門学校
デザインカレッジ



いよいよモジュールの分割に入る。

最初はなるべくコード量の少ない機能から試そう。

ということで、一番最初に作った「頂点を伸ばす」オペレータをモジュール分割してみる。

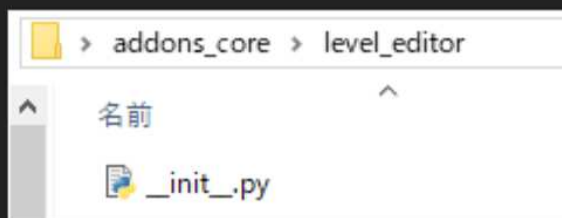
（なければ別のシンプルなオペレータでも構わない）

フォルダ内に「stretch_vertex.py」を新規作成する。
このファイル名から「.py」を抜いたものがそのままモジュール名になる。

モジュール分離

形式だけ適用する

日本工学院専門学校
デザインカレッジ



ファイル分割の前に、まずは1ファイル方式からフォルダ方式に変更してみよう。

1. level_editor.py があったフォルダに同じ名前の「level_editor」フォルダを作成し
2. フォルダ内にlevel_editor.pyを移動し
3. __init__.py にリネームする

ここまでやったら、一旦アドオンが元通り動作することを確認しておくこと。（Blender再起動とかもする）

この後のモジュール分割作業についても
少し変更を加えるごとに細かく動作確認するとよい。
細かく確認するのは面倒に感じるかもしれないが、
結局はの方が早く終わる。

形式だけ適用する

__init__.py

日本工学院専門学校
デザインカレッジ

__init__.py -io_curve_svg

```
# アドオン情報
bl_info = {
    ...
}

# モジュールのインポート
...

# クラスやグローバル関数の定義
...

def register():
    ...

def unregister():
    ...
```

左図は io_curve_svg アドオンの __init__.py の構成をまとめたものだ。

今まで1ファイルで書いてきたアドオンのスクリプトと特に変わらないということが分かるはずだ。

では何が違うのかというと、

- フォルダ内に複数のスクリプトファイルを置いて
- __init__.py からインポートしてクラスや関数を使用する

という点が異なる。

今まで我々は python の組み込みモジュールや Blender Python API の機能のみインポートしていたが、これに自作モジュールが加わるということだ。

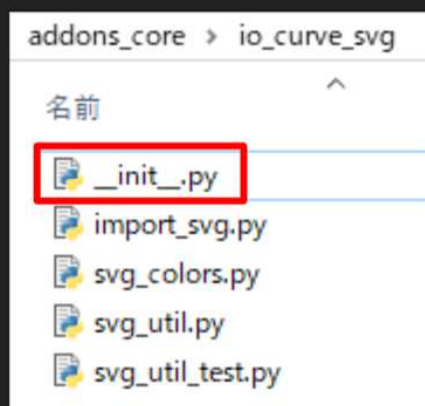
__init__.py

さらに各アドオンのフォルダを覗いてみると

- 1つの「__init__.py」ファイル
- 複数の任意の名前の .py ファイル

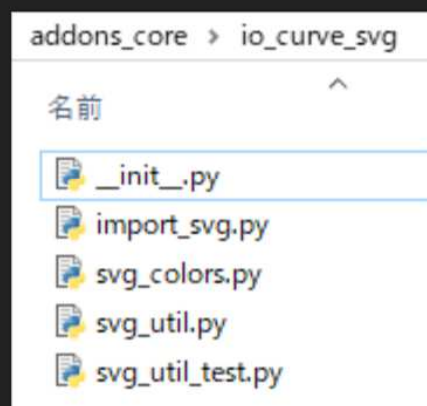
で構成されている。

ちょっと「__init__.py」をテキストエディタで開いて、中身を確認してみよう。



ファイル構成

日本工学院専門学校
デザインカレッジ



アドオンのフォルダを見てみると、既存のほとんどのアドオンはフォルダに格納されており、それぞれが複数の *.py ファイルで構成されている。

例えば左図は、io_curve_svg アドオンのフォルダだ。

(Blenderのバージョンによっては同じアドオンはないかもしれない)

ファイル構成
